Attention Is All You Need

你只需要注意力

Ashish Vaswani* Noam Shazeer Niki Parmar Jakob Uszkoreit
Llion Jones Aidan N. Gomez Łukasz Kaiser Illia Polosukhin

在注明出处的前提下，Google 在此授权，允许仅出于新闻报道或学术著作之目的复制本论文中的表格与插图。

Abstract

占主导地位的序列转导模型建立在复杂的循环或卷积神经网络之上，这些网络都包含一个编码器和一个解码器。表现最好的模型还通过注意力机制将编码器和解码器连接起来。我们提出了一种全新的、简洁的网络架构——**Transformer**，它完全基于注意力机制，彻底摒弃了循环与卷积。在两个机器翻译任务上的实验表明，这类模型在质量上更胜一筹，同时具备更强的可并行性，并且所需的训练时间大幅缩短。我们的模型在 WMT 2014 英译德翻译任务上取得 28.4 BLEU，相比此前的最佳结果（包括集成模型）提升了 2 BLEU 以上。在 WMT 2014 英译法翻译任务上，我们的模型在 8 块 GPU 上训练 3.5 天后，便创下了 41.8 的单模型 SOTA BLEU 纪录——这仅仅是文献中最佳模型训练成本的一小部分。我们还把 Transformer 成功应用于英语成分句法分析任务（无论训练数据规模大小），证明它具有良好的泛化能力。

1 引言

循环神经网络，尤其是长短期记忆网络 [13] 与门控循环神经网络 [7]，已被公认为序列建模与转导问题（如语言建模与机器翻译 [35，2，5]）中的最佳方法。此后的大量研究继续推动循环语言模型与编码器-解码器架构的边界 [38，24，15]。

循环模型通常沿输入和输出序列的符号位置来组织计算。它将位置与计算时间步对齐，生成一个隐藏状态序列 h_t，每个 h_t 是前一个隐藏状态 h_{t-1} 与位置 t 处输入的函数。这种固有的串行特性使得训练样本内部难以并行化，当序列变长时这一问题尤为关键，因为内存约束限制了跨样本的批处理。

近期的研究通过分解技巧 [21] 与条件计算 [32] 在计算效率上取得了显著提升，后者还同时改善了模型性能。然而，串行计算这一根本约束依然存在。

注意力机制已成为各类任务中令人信服的序列建模与转导模型不可或缺的一部分，它允许在建模依赖关系时不考虑它们在输入或输出序列中的距离 [2，19]。然而在除少数情形 [27] 之外的大多数情况下，这些注意力机制都是与循环网络结合使用的。

在本工作中，我们提出 **Transformer**——一种摒弃循环、完全依赖注意力机制在输入与输出之间建立全局依赖关系的模型架构。Transformer 允许更大幅度的并行化，并在 8 块 P100 GPU 上仅需训练十二小时即可达到翻译质量的新 SOTA。

*同等贡献，作者顺序随机。

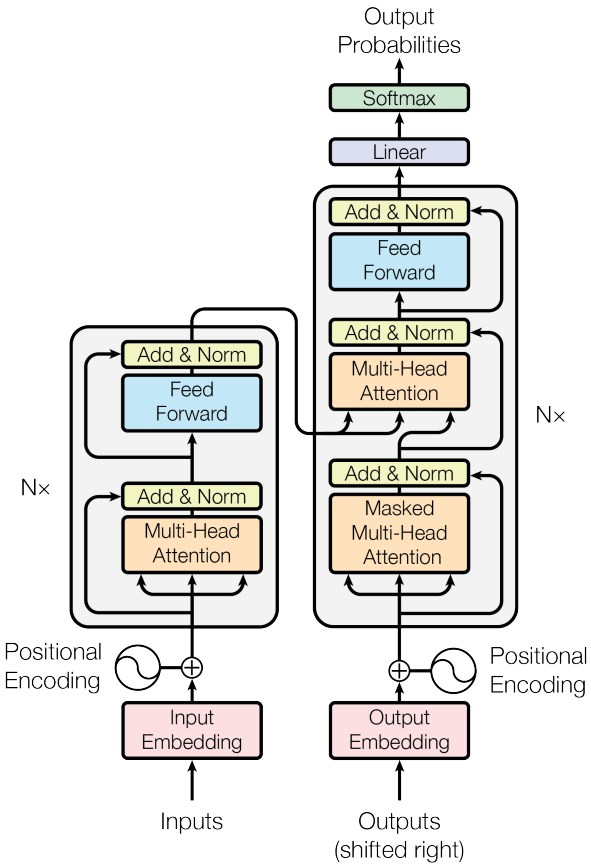


Figure 1: Transformer ——模型架构。

2 背景

减少串行计算这一目标，也是 Extended Neural GPU [16]、ByteNet [18] 和 ConvS2S [9] 的基础。它们都以卷积神经网络为基本构件，并行地为所有输入与输出位置计算隐藏表示。在这些模型中，关联来自任意两个输入或输出位置信号所需的运算次数，随位置间距而增长——ConvS2S 中是线性增长，ByteNet 中是对数增长。这使得学习远距离位置之间的依赖关系更加困难 [12]。而在 Transformer 中，这一开销被降低为常数次运算，代价是由于对注意力加权后的位置做平均而导致的等效分辨率下降，我们用第 3.2 节描述的多头注意力来抵消这一影响。

自注意力（self-attention），有时也称内部注意力（intra-attention），是一种将单一序列中不同位置关联起来以计算该序列表示的注意力机制。自注意力已成功应用于多种任务，包括阅读理解、生成式摘要、文本蕴含以及与任务无关的句子表示学习 [4, 27, 28, 22]。

端到端记忆网络（end-to-end memory networks）基于循环注意力机制而非与序列对齐的循环结构，已被证明在简单语言的问答与语言建模任务上表现良好 [34]。

然而据我们所知，Transformer 是第一个完全依靠自注意力来计算其输入与输出表示、而不使用与序列对齐的 RNN 或卷积的转导模型。在以下各节中，我们将描述 Transformer，阐述采用自注意力的动机，并讨论其相对于诸如 [17, 18] 与 [9] 等模型的优势。

3 模型架构

大多数有竞争力的神经序列转导模型都具有编码器-解码器结构 [5，2，35]。在这里，编码器将符号表示的输入序列 $(x_1, ..., x_n)$ 映射为连续表示序列 $\mathbf{z} = (z_1, ..., z_n)$。给定 $\mathbf{z}$，解码器随后每次生成一个元素，最终产生符号的输出序列 $(y_1, ..., y_m)$。在每一步中，模型都是自回归的 [10]，即在生成下一个符号时，会把此前已生成的符号作为额外输入。

Transformer 遵循这一总体架构，其编码器与解码器均由堆叠的自注意力层和逐位置的全连接层构成，分别对应图 1 的左半部分与右半部分。

3.1 编码器与解码器堆叠

编码器： 编码器由 $N = 6$ 个相同层堆叠而成。每层包含两个子层：第一个是多头自注意力机制；第二个是简单的、逐位置的全连接前馈网络。我们在两个子层周围各使用一个残差连接 [11]，随后接层归一化 [1]。也就是说，每个子层的输出为 $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$，其中 $\mathrm{Sublayer}(x)$ 是子层本身实现的函数。为便于这些残差连接，模型中的所有子层以及嵌入层，其输出维度均为 $d_{\mathrm{model}} = 512$。

解码器： 解码器同样由 $N = 6$ 个相同层堆叠而成。除了编码器每层中的两个子层之外，解码器还插入了第三个子层，用于对编码器堆叠的输出执行多头注意力。与编码器类似，我们在每个子层周围使用残差连接并随后做层归一化。我们还将解码器堆叠中的自注意力子层加以修改，以防止每个位置去关注其后续位置。这种掩码与输出嵌入向后偏移一位相结合，确保了位置 i 的预测只能依赖于 i 之前那些已知输出。

3.2 注意力

注意力函数可以描述为：将一个查询（query）与一组键值对（key-value pairs）映射到一个输出，其中查询、键、值与输出都是向量。输出是值的加权和，而分配给每个值的权重则由查询与对应键的兼容性函数计算得到。

3.2.1 缩放点积注意力

我们将所采用的特定注意力称为“缩放点积注意力”（Scaled Dot-Product Attention，图 2）。输入包括维度为 d_k 的查询和键，以及维度为 d_v 的值。我们计算查询与所有键的点积，每个除以 $\sqrt{d_k}$，并应用 softmax 函数来得到作用在值上的权重。

在实际中，我们同时在一组查询上计算注意力函数，把它们打包成矩阵 Q。键和值也被分别打包成矩阵 K 与 V。输出矩阵按如下方式计算：

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

最常用的两种注意力函数是加性注意力 [2] 和点积（乘性）注意力。点积注意力与我们的算法相同，仅差一个缩放因子 $\frac{1}{\sqrt{d_k}}$。加性注意力使用一个带单隐藏层的前馈网络来计算兼容性函数。两者在理论复杂度上相近，但点积注意力在实践中要快得多，也更节省空间，因为它可以用高度优化的矩阵乘法代码来实现。

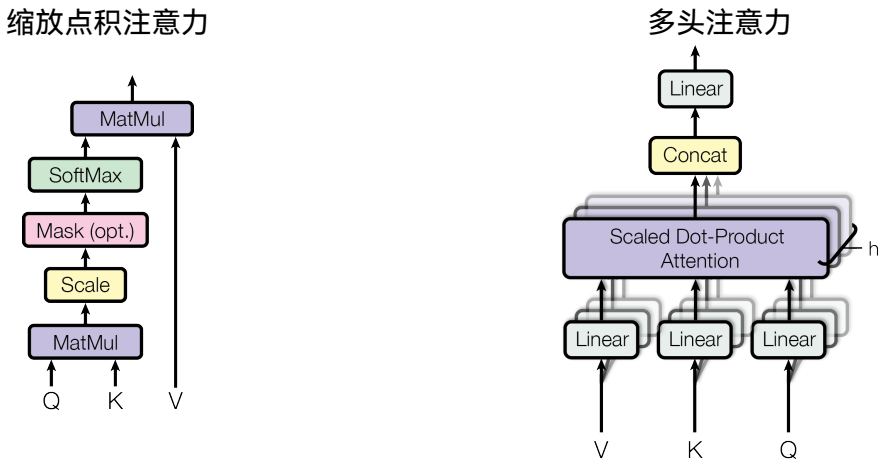


Figure 2: （左）缩放点积注意力。（右）多头注意力由若干并行运行的注意力层组成。

虽然当 d_k 较小时这两种机制表现相当，但在 d_k 较大且不做缩放时，加性注意力优于点积注意力 [3]。我们怀疑，对于较大的 d_k，点积在数值上变得很大，会把 softmax 推入梯度极小的区域[1]。为抵消这一影响，我们将点积乘以 $\frac{1}{\sqrt{d_k}}$ 进行缩放。

3.2.2 多头注意力

与其用 d_{model} 维的键、值与查询执行单一的注意力函数，我们发现：将查询、键、值用不同且可学习的线性投影分别投影 h 次到 d_k、d_k、d_v 维是有益的。随后我们在这些投影后的查询、键、值上并行执行注意力函数，得到 d_v 维的输出值，将其拼接后再做一次投影，便得到最终结果，如图 2 所示。

多头注意力允许模型在不同位置联合关注来自不同表示子空间的信息。若只有一个注意力头，做平均会抑制这种能力。

$$\begin{aligned}\text{MultiHead}(Q,K,V) &= \text{Concat}(\text{head}_1,\ldots,\text{head}_h)\,W^O\\ \text{where head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)\end{aligned}$$

其中投影是参数矩阵 $W_i^Q \in \mathbb{R}^{d_{\text{model}}\times d_k}$，$W_i^K \in \mathbb{R}^{d_{\text{model}}\times d_k}$，$W_i^V \in \mathbb{R}^{d_{\text{model}}\times d_v}$，$W^O \in \mathbb{R}^{hd_v\times d_{\text{model}}}$。

在本工作中，我们采用 $h=8$ 个并行的注意力层（即头）。对每个头，我们取 $d_k = d_v = d_{\text{model}}/h = 64$。由于每个头的维度降低，其总计算量与满维度的单头注意力相当。

3.2.3 注意力在本模型中的应用

Transformer 以三种不同方式使用多头注意力：

- 在“编码器-解码器注意力”层中，查询来自前一个解码器层，而记忆的键和值来自编码器的输出。这使得解码器中的每个位置都能关注输入序列中的所有位置。这与序列到序列模型（例如 [38, 2, 9]）中典型的编码器-解码器注意力机制一致。

[1] 为说明点积为何变大，假设 q 和 k 的各分量是均值 0、方差 1 的独立随机变量，那么它们的点积 $q\cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的均值为 0、方差为 d_k。

- 编码器中包含自注意力层。在自注意力层中，所有的键、值与查询都来自同一个地方——本例中即编码器中前一层的输出。编码器中的每个位置都可以关注编码器前一层的所有位置。
- 类似地，解码器中的自注意力层允许解码器中每个位置关注该位置及其之前的所有解码器位置。我们需要阻止解码器中向左的信息流动以保持自回归特性。我们在缩放点积注意力内部，通过将 softmax 输入中所有对应非法连接的值置为 $-\infty$ 来实现掩码。参见图 2。

3.3 逐位置前馈网络

除了注意力子层之外，我们的编码器与解码器中的每一层都包含一个全连接前馈网络，它被分别且相同地施加于每个位置。它由两次线性变换及其中间的 ReLU 激活组成。

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2}$$

虽然不同位置上的线性变换是相同的，但它们在不同层之间使用不同的参数。另一种描述方式是把它看作两个核大小为 1 的卷积。输入与输出维度为 $d_{\mathrm{model}} = 512$，而内层维度为 $d_{ff} = 2048$。

3.4 嵌入与 Softmax

与其他序列转导模型类似，我们使用可学习的嵌入，将输入词元与输出词元转换为 d_{model} 维的向量。我们也使用常规的可学习线性变换与 softmax 函数，将解码器输出转换为对下一个词元预测概率。在我们的模型中，两个嵌入层与 softmax 前的线性变换共享同一个权重矩阵，类似于 [30]。在嵌入层中，我们将这些权重乘以 $\sqrt{d_{\mathrm{model}}}$。

3.5 位置编码

由于我们的模型既不包含循环也不包含卷积，为了让模型能够利用序列的顺序，我们必须注入一些关于序列中词元相对或绝对位置的信息。为此，我们在编码器与解码器堆叠底部的输入嵌入之上，加入“位置编码”。位置编码与嵌入具有相同的维度 d_{model}，以便二者可以相加。位置编码有多种选择，既可以是可学习的，也可以是固定的 [9]。

在本工作中，我们使用不同频率的正弦和余弦函数：

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\mathrm{model}}})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\mathrm{model}}})$$

其中 pos 是位置，i 是维度。也就是说，位置编码的每一维都对应一个正弦波。其波长从 2π 到 $10000 \cdot 2\pi$ 呈几何级数。我们之所以选择这个函数，是因为我们推测它能让模型容易地学会按相对位置进行关注：对于任意固定偏移量 k，PE_{pos+k} 都可以表示为 PE_{pos} 的线性函数。

我们也尝试过使用可学习的位置嵌入 [9] 来替代它，并发现两种版本产生的结果几乎完全一致（见表 3 第（E）行）。我们最终选择正弦版本，是因为它可能允许模型外推到比训练中所见更长的序列长度。

Table 1: 不同层类型的最大路径长度、每层复杂度与最少串行运算次数。n 为序列长度，d 为表示维度，k 为卷积核大小，r 为受限自注意力中邻域的大小。

层类型	每层复杂度	串行运算次数	最大路径长度
自注意力	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
循环	$O(n \cdot d^2)$	$O(n)$	$O(n)$
卷积	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
自注意力（受限）	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

4 为何选择自注意力

在本节中，我们将自注意力层与常用于把变长符号表示序列 $(x_1, ..., x_n)$ 映射为等长序列 $(z_1, ..., z_n)$（其中 $x_i, z_i \in \mathbb{R}^d$，例如典型序列转导编码器或解码器中的某一隐藏层）的循环层和卷积层，在各个方面进行比较。出于采用自注意力的动机，我们考虑了三个目标。

其一是每层的总计算复杂度。

其二是可以被并行化的计算量，以所需的最少串行运算次数来衡量。

其三是网络中长程依赖之间的路径长度。学习长程依赖是许多序列转导任务中的关键挑战。影响学习此类依赖能力的一个关键因素，是前向与反向信号在网络中所需穿越路径的长度。输入与输出序列任意位置之间的这种路径越短，就越容易学到长程依赖 [12]。因此我们还比较了由不同层类型构成的网络中，任意两个输入与输出位置之间的最大路径长度。

如表 1 所示，自注意力层仅用常数次串行运算即可将所有位置连接起来，而循环层需要 $O(n)$ 次串行运算。

就计算复杂度而言，当序列长度 n 小于表示维度 d 时，自注意力层比循环层更快——而这在机器翻译中最先进的模型所用的句子表示（例如词片 [38] 与字节对 [31] 表示）中通常是成立的。为了改善涉及超长序列任务的计算性能，可以将自注意力限制为只考虑以相应输出位置为中心、大小为 r 的邻域。这会把最大路径长度增加到 $O(n/r)$。我们计划在未来的工作中进一步研究这一思路。

单个核宽度为 $k < n$ 的卷积层无法连接所有输入与输出位置对。要做到这一点，在连续核的情况下需要 $O(n/k)$ 层卷积，在膨胀卷积的情况下需要 $O(log_k(n))$ 层 [18]，这都会增加网络中任意两个位置间最长路径的长度。

卷积层通常比循环层更昂贵，差距是一个因子 k。然而可分离卷积 [6] 会显著降低复杂度，降至 $O(k \cdot n \cdot d + n \cdot d^2)$。即便取 $k = n$，可分离卷积的复杂度也恰好等于一个自注意力层加上一个逐位置前馈层的组合，而这正是我们模型所采用的方式。

作为附带的好处，自注意力或许能产生更具可解释性的模型。我们检查了模型中的注意力分布，并在附录中给出并讨论了一些示例。不仅单个注意力头清楚地学会了执行不同任务，许多注意力头还表现出与句子句法与语义结构相关的行为。

5 训练

本节描述我们模型的训练方案。

5.1 训练数据与批处理

我们在标准的 WMT 2014 英德数据集上训练，该数据集包含约 450 万个句子对。句子采用字节对编码 [3] 进行编码，其共享的源-目标词表大约包含 37000 个词元。对于英法任务，我们使用了规模显著更大的 WMT 2014 英法数据集，包含 3600 万个句子，并将词元切分到一个 32000 词元的词片词表 [38] 中。句子对按近似序列长度进行批处理。每个训练批次包含一组句子对，约 25000 个源词元和 25000 个目标词元。

5.2 硬件与时间安排

我们在一台配备 8 块 NVIDIA P100 GPU 的机器上训练模型。对于使用本文所述超参数的基础模型，每个训练步约耗时 0.4 秒。我们对基础模型总共训练了 100,000 步，即 12 小时。对于大模型（见表 3 最后一行），每步耗时为 1.0 秒，共训练 300,000 步（3.5 天）。

5.3 优化器

我们使用 Adam 优化器 [20]，参数为 $\beta_1 = 0.9$、$\beta_2 = 0.98$、$\epsilon = 10^{-9}$。我们在训练过程中按如下公式调整学习率：

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \tag{3}$$

也就是说，在前 $warmup_steps$ 个训练步内学习率线性增长，此后按步数倒数的平方根比例下降。我们取 $warmup_steps = 4000$。

5.4 正则化

我们在训练中使用了三种正则化：

残差 Dropout 我们在每个子层的输出（在与子层输入相加并归一化之前）应用 dropout [33]。此外，我们也在编码器与解码器堆叠中对嵌入与位置编码之和应用 dropout。对于基础模型，我们使用丢弃率 $P_{drop} = 0.1$。

标签平滑 在训练中，我们采用了取值为 $\epsilon_{ls} = 0.1$ 的标签平滑 [36]。这会损害困惑度（因为模型学得更“不确定”），但能提升准确率与 BLEU 分数。

6 结果

6.1 机器翻译

在 WMT 2014 英译德翻译任务上，大 Transformer 模型（表 2 中的 Transformer (big)）相比此前已报道的最佳模型（包括集成模型）高出 2.0 BLEU 以上，刷新了 28.4 的 SOTA BLEU 成绩。该模型的配置列于表 3 最后一行。训练在 8 块 P100 GPU 上耗时 3.5 天。即便是我们的基础模型，也以远低于任何有竞争力模型的训练成本，超越了所有此前公开发表的模型与集成。

在 WMT 2014 英译法翻译任务上，我们的大模型取得 41.0 的 BLEU 分数，超越了此前所有公开发表的单模型，而其训练成本不到此前 SOTA 模型的 $1/4$。用于英译法的 Transformer (big) 模型使用了丢弃率 $P_{drop} = 0.1$，而非 0.3。

Table 2：在 English-to-German 与 English-to-French 的 newstest2014 测试上，Transformer 以远低于以往模型的训练成本取得了更优的 BLEU 分数。

模型	BLEU		训练成本（FLOPs）	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk 集成 [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL 集成 [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S 集成 [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer（基础模型）	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer（大模型）	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

对于基础模型，我们使用的是对最后 5 个检查点取平均所得到的单一模型，这些检查点以 10 分钟为间隔写入。对于大模型，我们对最后 20 个检查点取平均。我们使用束搜索，束大小为 4，长度惩罚 $\alpha = 0.6$ [38]。这些超参数是在开发集上经实验后选定的。我们在推理时把最大输出长度设为输入长度 + 50，但在可能时提前终止 [38]。

表 2 总结了我们的结果，并将我们的翻译质量与训练成本与文献中的其他模型架构进行了对比。我们通过将训练时间、所用 GPU 数量与每块 GPU 持续单精度浮点算力的估计值相乘，来估计训练一个模型所用的浮点运算次数[2]。

6.2 模型变体

为了评估 Transformer 不同组件的重要性，我们以不同方式改变了基础模型，并在开发集 newstest2013 上测量英译德翻译性能的变化。我们采用了上一节描述的束搜索，但不做检查点平均。结果见表 3。

在表 3 的（A）行中，我们在保持计算量不变的前提下（如第 3.2.2 节所述）改变了注意力头的数量以及注意力键和值的维度。单头注意力比最佳设置差 0.9 BLEU，但头数过多时质量也会下降。

在表 3 的（B）行中，我们观察到减小注意力键的尺寸 d_k 会损害模型质量。这表明确定兼容性并不容易，比点积更复杂的兼容性函数或许是有益的。我们还在（C）行与（D）行中进一步观察到，正如预期，更大的模型更好，而 dropout 对避免过拟合很有帮助。在（E）行中，我们用可学习的位置嵌入 [9] 替代了正弦位置编码，并观察到与基础模型几乎相同的结果。

6.3 英语成分句法分析

为了评估 Transformer 是否能泛化到其他任务，我们在英语成分句法分析上做了实验。该任务带来特定挑战：输出受到强结构约束，且显著长于输入。

此外，RNN 序列到序列模型在小数据场景下一直无法取得 SOTA 结果 [37]。

[2] 对于 K80、K40、M40 与 P100，我们分别采用了 2.8、3.7、6.0 与 9.5 TFLOPS。

Table 3: Transformer 架构上的变体。未列出的值与基础模型相同。所有指标均在英译德开发集 newstest2013 上测得。所列困惑度为按我们的字节对编码计算的每词片困惑度，不应与每词困惑度相比较。

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	训练步数 steps	PPL (dev)	BLEU (dev)	参数量 $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	用位置嵌入替代正弦位置编码									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

我们在 Penn Treebank [25] 的 Wall Street Journal (WSJ) 部分上训练了一个 4 层、$d_{model} = 1024$ 的 Transformer，约 4 万训练句子。我们还在半监督设定下训练了它，使用了规模更大、约 1700 万句子的高置信度与 BerkeleyParser 语料 [37]。对仅 WSJ 设定我们使用 16K 词元的词表，对半监督设定使用 32K 词元的词表。

我们仅进行了少量实验，在第 22 节开发集上选择 dropout（包括注意力与残差，第 5.4 节）、学习率与束大小，其他所有参数均与英译德基础翻译模型保持一致。在推理时，我们把最大输出长度增加到输入长度 + 300。无论仅 WSJ 还是半监督设定，我们都使用束大小 21、$\alpha = 0.3$。

我们在表 4 中的结果显示，尽管缺少针对任务的调优，我们的模型依然表现出人意料地好，除循环神经网络文法 [8] 之外，它优于所有此前报道的模型。

与 RNN 序列到序列模型 [37] 不同，Transformer 即便只在 4 万句的 WSJ 训练集上训练，也能胜过 BerkeleyParser [29]。

Table 4: Transformer 在英语成分句法分析任务上具有良好的泛化能力（结果在 WSJ 第 23 节上测得）

分析器	训练设定	**WSJ 23 F1**
Vinyals & Kaiser el al. (2014) [37]	仅 WSJ，判别式	88.3
Petrov et al. (2006) [29]	仅 WSJ，判别式	90.4
Zhu et al. (2013) [40]	仅 WSJ，判别式	90.4
Dyer et al. (2016) [8]	仅 WSJ，判别式	91.7
Transformer (4 层)	仅 WSJ，判别式	91.3
Zhu et al. (2013) [40]	半监督	91.3
Huang & Harper (2009) [14]	半监督	91.3
McClosky et al. (2006) [26]	半监督	92.1
Vinyals & Kaiser el al. (2014) [37]	半监督	92.1
Transformer (4 层)	半监督	92.7
Luong et al. (2015) [23]	多任务	93.0
Dyer et al. (2016) [8]	生成式	93.3

7 结论

在本工作中，我们提出了 **Transformer**——第一个完全基于注意力的序列转导模型，用多头自注意力替换了编码器-解码器架构中最常用的循环层。

对于翻译任务，Transformer 的训练速度远快于基于循环层或卷积层的架构。在 WMT 2014 英译德和英译法两个翻译任务上，我们都刷新了当时的最佳成绩。在前者上，我们最好的模型甚至超越了此前所有已报道的集成模型。

我们对基于注意力的模型的未来充满期待，并计划将其应用到其他任务上。我们计划将 Transformer 拓展到涉及文本以外输入输出模态的问题上，并研究局部、受限的注意力机制，以高效地处理诸如图像、音频与视频等大型输入与输出。让生成过程更具并行性、减少串行依赖，是我们另一个研究目标。

我们用于训练和评估模型的代码可在 https://github.com/tensorflow/tensor2tensor 获取。

致谢　我们感谢 Nal Kalchbrenner 与 Stephan Gouws 提供的富有启发性的意见、纠正与灵感。

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.

[3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive explo-

ration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.

[4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.

[5] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.

[6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.

[7] Junyoung Chung, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.

[8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proc. of NAACL*, 2016.

[9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.

[10] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.

[13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.

[15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.

[16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.

[17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.

[18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.

[19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.

[20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.

[22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.

[23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Lukasz Kaiser. Multi-task sequence to sequence learning. *arXiv preprint arXiv:1511.06114*, 2015.

[24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.

[25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.

[26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159. ACL, June 2006.

[27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.

[28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.

[29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.

[30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.

[31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.

[32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.

[35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.

[36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.

[37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, 2015.

[38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

[39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.

[40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

注意力可视化

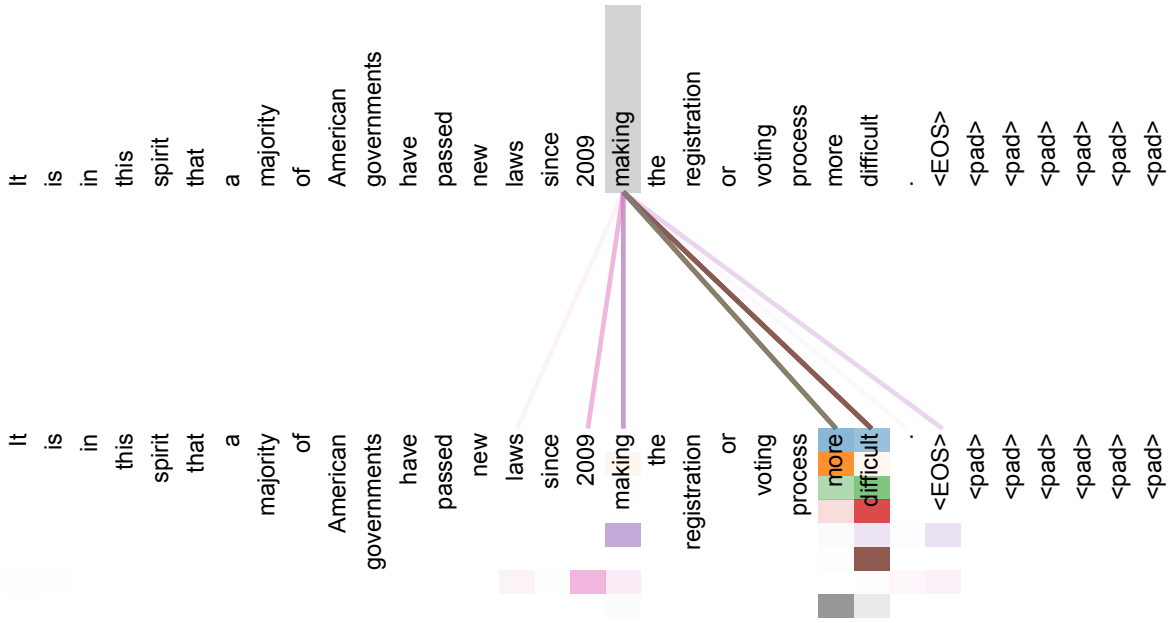


Figure 3: 在第 5 层（共 6 层）编码器自注意力中，注意力机制追踪长距离依赖的一个示例。许多注意力头关注动词“making”的远距离依赖，补全了短语“making...more difficult”。此处仅展示单词“making”的注意力。不同颜色代表不同的头。建议彩色查看。

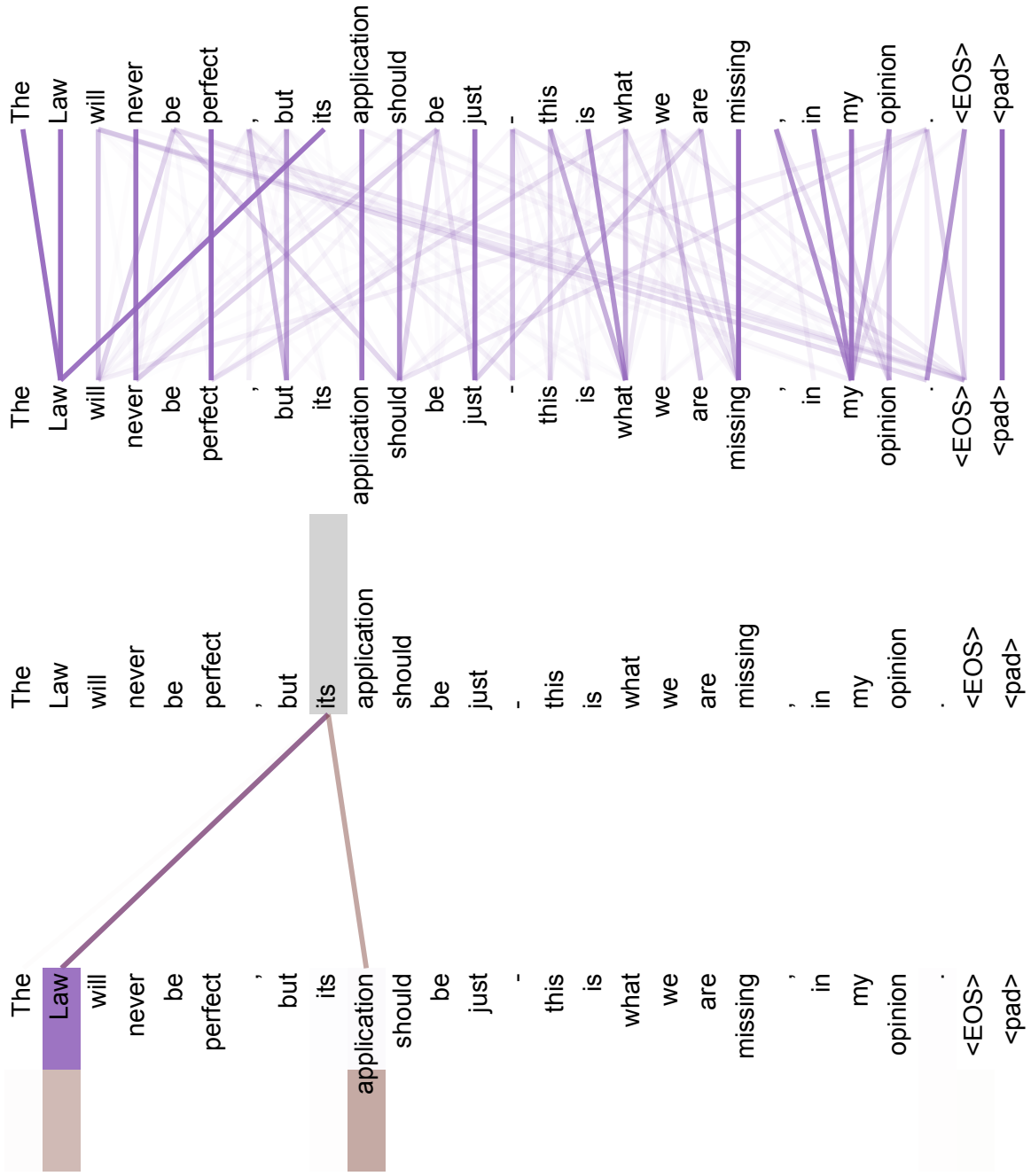


Figure 4: 同样在第 5 层（共 6 层）中的两个注意力头，看上去参与了指代消解。上：第 5 个头的完整注意力。下：仅就单词“its”而言，第 5 与第 6 个注意力头的孤立注意力。注意该词的注意力非常尖锐。

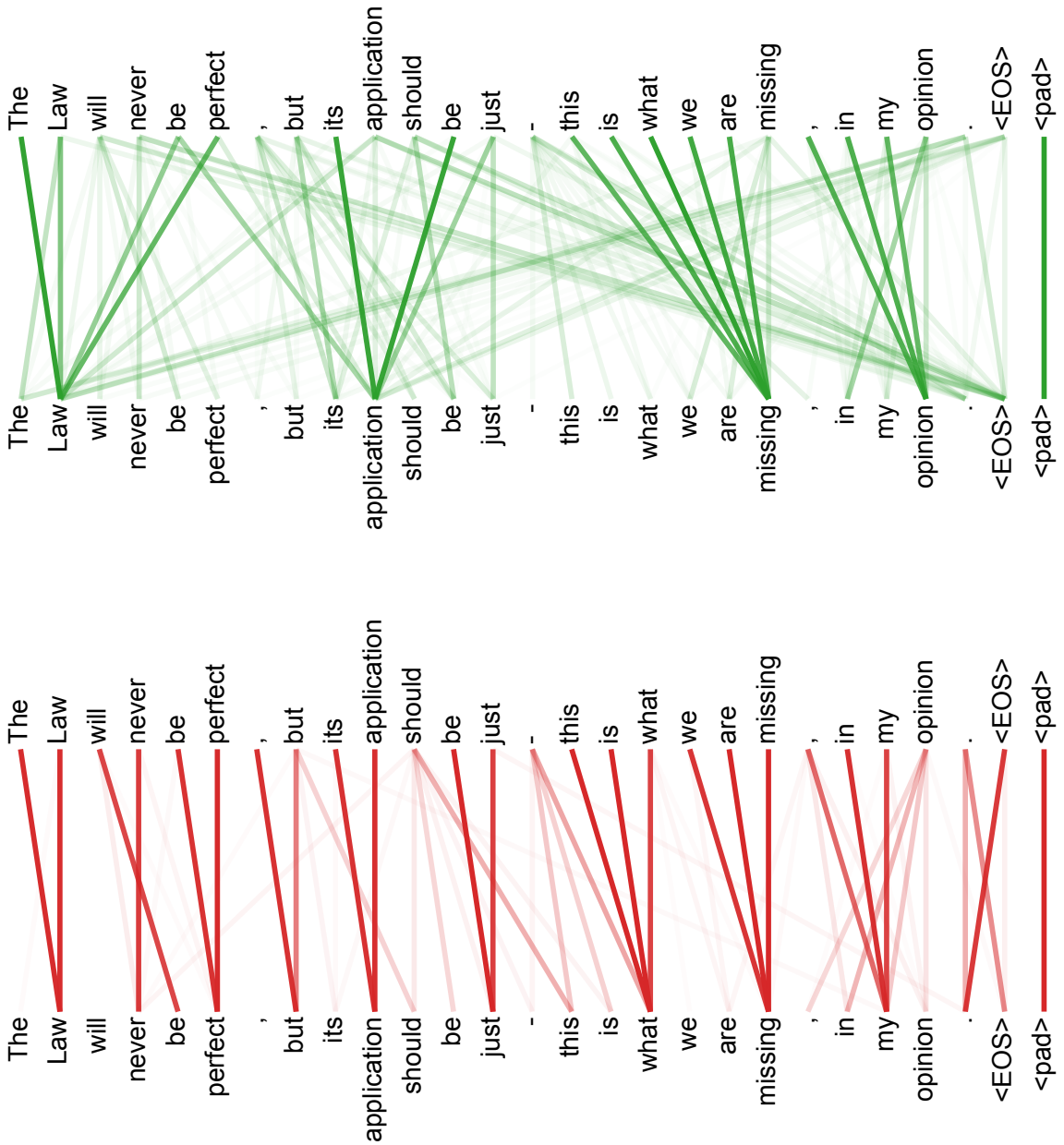


Figure 5: 许多注意力头表现出似乎与句子结构相关的行为。我们在上方给出两个这样的示例，分别来自第 5 层（共 6 层）编码器自注意力中两个不同的头。这些头清楚地学会了执行不同的任务。